Checkpoint One Report

CIS4650 | Hisham Issa | 1194466 | hissa01@uoguelph.ca

Introduction

For this checkpoint, I built the front-end of a compiler for the C- programming language. The goal was to create a working scanner, parser, and error recovery system that takes C- source code and turns it into an abstract syntax tree. I used JFlex for the scanner and CUP for the parser. This report covers what I built, the decisions I made, the problems I encountered, and what I learned.

Scanner Implementation

The scanner reads the source file character by character and groups them into tokens. I used JFlex as required by the assignment, which simplified things a lot compared to hand-writing a scanner.

I defined all the C- token types: keywords (if, while, return), operators (+, -, ==), identifiers, and numbers. The tricky part was ensuring multi-character operators like <= and == got recognized before their single-character versions. I had to put those rules first in the flex file, otherwise <= would scan as < followed by =.

For identifiers, I used the pattern `[a-zA-Z_][a-zA-Z0-9_]*`, meaning they start with a letter or underscore, then can have letters, digits, or underscores. Numbers are just `[0-9]+` since C- only has integers.

Comments were interesting to handle. C- uses C-style `/* */` comments that can't be nested. I used the pattern `"/*" [^*] ~"*/" | "/*" "*"+ "/"` which handles regular comments and edge cases like `/**/` or `/***/`. The key was making sure the scanner doesn't interpret anything inside comments.

Position tracking was essential for error reporting. JFlex's `%line` and `%column` directives let me track where each token actually appears. When creating tokens, I pass `yyline` and `yycolumn` so errors can report "Error on line 5, column 12" instead of vague messages.

The main challenge was getting the comment pattern right. Initially, it was matching too much. Testing it with the C- programs we were provided helped me catch these issues early.

Parser Implementation

The C- specification provides a clean grammar, but it has multiple levels of non-terminals for expressions to handle precedence and associativity. The assignment required us to simplify this using CUP. Instead of separate non-terminals for obool-expression, abool-expression, simple-expression, additive-expression, and term, I combined these into fewer rules with precedence declarations:

```
precedence right ASSIGN;
precedence left OR;
precedence left AND;
precedence right NOT;
precedence nonassoc LT, LE, GT, GE, EQEQ, NE;
precedence left PLUS, MINUS;
precedence left TIMES, OVER;
```

This tells CUP how to resolve ambiguities without extra grammar layers. The `nonassoc` for comparison operators means you can't chain them, `a < b < c` is a syntax error, matching C- semantics.

I designed the AST node classes based on C-'s language needs. There's a base `Absyn` class, then abstract classes for major categories: `Dec` (declarations), `Exp` (expressions), and `Var` (variable references). The concrete classes cover all C- constructs: `SimpleDec`, `ArrayDec`, `FunctionDec`, `FunctionProtoDec` for declarations. `SimpleVar` and `IndexVar` for variables. `IntExp`, `BoolExp`, `VarExp`, `CallExp`, `OpExp`, `AssignExp`, `IfExp`, `WhileExp`, `ReturnExp`, and `CompoundExp` for expressions. And `DecList`, `VarDecList`, `ExpList` for sequences.

For displaying the AST, I used the visitor pattern. Each AST node has an accept method, and `ShowTreeVisitor` implements how to display each type. This separates display logic from AST structure. The visitor uses indentation (4 spaces per level) to show tree structure and prints useful info like function names, variable names, and operator types.

The dangling else problem appeared as expected. When parsing '`if (E1) if (E2) S1 else S2`', the else could attach to either if. CUP defaults to shift, which matches `else` to the nearest `if`. I told CUP to expect 5 conflicts total (including error recovery) with `-expect 5` in the `Makefile`.

Error Recovery Implementation

Error recovery was the most challenging part in my opinion. The goal is to detect syntax errors but continue to parse and find more errors instead of stopping at the first one. I used CUP's error token to add error productions at key synchronization points.

My approach was that when the parser hits something unexpected, it triggers an error production that skips tokens until it finds a synchronization point (semicolons, braces, parentheses), reports a specific error message with line and column numbers, and then continues parsing.

I added error productions at multiple grammar levels.

At the **statement level**, we have `statement_list ::= statement_list error SEMI` and `expression_stmt ::= error SEMI` that catch invalid statements and skip to the next semicolon.

For **control flow**, there is `selection_stmt ::= IF LPAREN error RPAREN statement` and `iteration_stmt ::= WHILE LPAREN error RPAREN statement` that catch malformed conditions, but still parse the body.

For **expressions**: `var ::= ID LBRACKET error RBRACKET` and `factor ::= LPAREN error RPAREN` catch errors inside array indexing and parentheses.

For **declarations**: `var_declaration ::= error SEMI` and `fun_declaration ::= type_specifier ID LPAREN error RPAREN compound_stmt` handle malformed declarations and parameter lists.

I customized the error messages to be as specific as possible. For example, "Invalid array index expression" for array indexing failures, "Invalid condition in if statement" for malformed if conditions, and so on. Each message includes line and column numbers. CUP normally prints unhelpful automatic messages like "instead expected token classes are...". I overrode the `syntax_error` and `unrecovered_syntax_error` methods to suppress these and show only my custom messages.

Error recovery presented some big challenges for me. Cascading errors sometimes caused the parser to report additional false errors. For example, a missing semicolon can make the next line look wrong. I minimized this with good synchronization points, but some cascading remains. Generic versus specific messages were another issue. Errors happening early get generic messages, while errors in specific contexts (inside if conditions) get targeted messages. This is

normal even in professional compilers. Adding error productions created additional shift/reduce conflicts, requiring me to update the Makefile to expect 5 conflicts instead of 1.

Testing

I created five test programs as required. **test/1.cm** is a clean program with no errors, including global variables, functions with parameters, recursion, control flow, and arrays, validating that the parser handles all C- features. **test/2.cm** has three statement-level errors from invalid tokens, testing basic error detection and recovery. **test/3.cm** has three syntax errors: missing expression in assignment, invalid statement with random identifiers, and missing semicolon, testing recovery across different statement types. **test/4.cm** demonstrates three specific error messages: invalid array index expression, invalid condition in if statement, and invalid condition in while loop, showing targeted feedback. **test/5.cm** is a stress test with 10+ errors, including malformed declarations, invalid expressions, missing punctuation, bad array indices, malformed function calls, and invalid control structures, testing continued parsing through cascading errors. All the test programs work as expected.

Lessons Learned

JFlex and CUP are powerful but picky. Small mistakes were causing confusing errors. I learned to check the warnings carefully. They often point to real problems even when tools still generate code.

File organization matters. I initially tried organizing everything into packages, but it got messy. Keeping it simple, like the Tiny example (only the absyn package), worked much better. Generated files go in the root directory, source files in src/, and compiled classes in bin/.

Testing incrementally is essential. I got the scanner working first, then added the parser without semantic actions, then added AST building, then error recovery. Each step confirmed previous work was solid.

Current Limitations and Future Work

The parser accepts syntactically valid C- programs but doesn't perform semantic checking. Variables declared before use, type matching in assignments, correct function call arguments, etc. That's for checkpoint two.

Some cascading errors are inevitable. When the parser gets really confused (like from a missing brace), it sometimes reports follow-up errors that aren't real.

The C- spec says `input()` and `output()` are predefined, but my compiler treats them as regular function calls with no special handling.

Conclusion

I now have a working front-end for C- that scans tokens, parses the grammar, builds a complete abstract syntax tree, and provides reasonable error recovery. The tools (JFlex and CUP) handled the heavy lifting, but getting details right took careful work.

The biggest lessons were keeping things simple, testing constantly, and not expecting perfection from error recovery. The parser correctly handles all C- language features, including functions, arrays, control flow, and expressions with proper precedence. It provides helpful error messages and continues parsing after errors to find multiple problems in one compilation.

The next step for checkpoint two is adding semantic analysis. Type checking, scope resolution, and verifying functions are called correctly.